



Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro



Lab Manual: Introduction to Django — Project Setup, Apps, and Models

Course: Robotics & Automatic Systems

Lab Topic: Setting up a Django development environment, creating a project, building an app, defining models, and using the Django Admin

Estimated Duration: 1.5 – 2 hours

Learning Objectives

By the end of this lab, students will be able to:

1. Create and manage a Python virtual environment.
 2. Install Django and verify the installation.
 3. Create a new Django project and run the development server.
 4. Create a Django app and register it with the project.
 5. Define a database model and understand basic field types.
 6. Generate and apply database migrations.
 7. Register a model with the Django Admin and create a superuser.
 8. Customize the admin representation of a model using `__str__`.
-

Prerequisites

- Python installed on your system (check with `python --version`)
- Basic understanding of Python syntax and OOP (classes)
- A code editor (VS Code recommended)
- Command line / terminal access



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Part 1: Environment Setup

Step 1.1 — Create a Virtual Environment

A virtual environment isolates your project's dependencies from your global Python installation.

Shell

```
python -m venv [environment_name]
```

Example:

Shell

```
python -m venv test-venv
```

Step 1.2 — Activate the Virtual Environment

Navigate to the **Scripts** folder inside your environment and run:

Shell

```
[path_to_Scripts_folder]\activate.bat
```

Note: Once activated, your terminal prompt will typically show the environment name in parentheses, e.g. **(test-venv)**.



Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro



Step 1.3 — Deactivate the Virtual Environment

You do not need to navigate back to the Scripts folder to deactivate. Simply run:

```
Shell  
deactivate
```

Step 1.4 — Install Packages with pip

pip is used to install packages into either the global Python installation or the active virtual environment.

```
Shell  
pip install django==5.2.15
```

Step 1.5 — List Installed Packages

To verify what's installed in your environment:

```
Shell  
pip list
```

✓ Checkpoint: Confirm Django appears in your **pip list** output before proceeding.



Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro



Part 2: Creating a Django Project

Step 2.1 — Create the Project

Shell

```
django-admin startproject [project_name]
```

Step 2.2 — Run the Development Server

Django projects are run using **manage.py**:

Shell

```
python manage.py runserver
```

Visit <http://127.0.0.1:8000/> in your browser to confirm the server is running.

Part 3: Creating an App and a Model

Step 3.1 — Create a New App

Shell

```
python manage.py startapp [app_name]
```

Example:



Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro



Shell

```
python manage.py startapp myapp
```

Step 3.2 — Register the App

Open **settings.py** and add your app's name to the **INSTALLED_APPS** list:

Python

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    # ... above are default apps  
    # ... below is our custom app  
    'myapp',  
]
```

Why this matters: Django won't recognize your app's models, templates, or migrations until it's listed in **INSTALLED_APPS**.



Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro



Step 3.3 — Define a Model

Open `myapp/models.py` and define your model:

```
Python
from django.db import models

# Create your models here.

class Person(models.Model):

    first_name = models.CharField()

    last_name = models.CharField(max_length=10)
```

Note on CharField: In modern Django versions, `CharField` requires a `max_length` argument in most cases. Pay attention to `first_name` above — it's missing `max_length`, which will raise an error. This is intentional for you to notice and fix (see Exercise 1 below).

Part 4: Migrations

Migrations translate your Python model definitions into actual database tables.

Step 4.1 — Generate Migrations



Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro



Shell

```
python manage.py makemigrations
```

This creates migration files describing the changes to apply to the database.

Step 4.2 — Apply Migrations

Shell

```
python manage.py migrate
```

This executes the migration files against the database.

Rule of thumb: Run these two commands together, in this order, every time you change a model.

Part 5: Django Admin

Step 5.1 — Register Your Model

Open **myapp/admin.py**:

Python

```
from django.contrib import admin  
  
from myapp.models import Person
```



Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro



```
# Register your models here.
```

```
admin.site.register(Person)
```

Step 5.2 — Create a Superuser

Shell

```
python manage.py createsuperuser
```

Follow the prompts to set a username, email, and password.

Example (for local testing only): username **admin**, password **admin**

⚠ Security Note: Never use weak credentials like **admin/admin** outside of local development.

Step 5.3 — Improve Admin Representation with `__str__`

By default, objects appear in the admin as **Person object (1)**. To make them human-readable, define `__str__`:

Python

```
from django.db import models
```

```
class Person(models.Model):
```

```
    first_name = models.CharField()
```




Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro



```
last_name = models.CharField(max_length=10)

def __str__(self):
    return self.first_name + " " + self.last_name
```

Now, instead of **Person object (1)**, entries will display as **John Doe**.

Full Workflow Summary (Quick Reference)

Step	Command
Create virtual env	<code>python -m venv test-venv</code>
Activate venv	<code>[Scripts_path]\activate.bat</code>
Deactivate venv	<code>deactivate</code>
Install Django	<code>pip install django==5.2.15</code>



Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro



List packages	<code>pip list</code>
Create project	<code>django-admin startproject [project_name]</code>
Run server	<code>python manage.py runserver</code>
Create app	<code>python manage.py startapp [app_name]</code>
Make migrations	<code>python manage.py makemigrations</code>
Apply migrations	<code>python manage.py migrate</code>
Create superuser	<code>python manage.py createsuperuser</code>

Exercises

1. Fix the Model: The `first_name` field in `Person` is missing `max_length`. Add an appropriate value and re-run migrations.
2. Extend the Model: Add an `email` field (`EmailField`) and an `age` field (`IntegerField`) to the `Person` model.



3. Admin Customization: Explore `list_display` in a custom `ModelAdmin` class to show `first_name`, `last_name`, and `email` as columns in the admin list view.
4. Test It: Create 3 `Person` records through the Django Admin panel and verify they display correctly using your `__str__` method.

Common Errors & Troubleshooting

Error	Likely Cause	Fix
<code>CharField()</code> missing <code>max_length</code>	Forgot <code>max_length</code> argument	Add <code>max_length=100</code> or similar
App not found / model not detected	App not added to <code>INSTALLED_APPS</code>	Add app name to settings.py
No changes detected on makemigrations	Model wasn't saved, or app not registered	Save file; check <code>INSTALLED_APPS</code>
Admin shows <code>Person</code> object (1)	Missing <code>__str__</code> method	Add <code>__str__</code> to the model